

SiNDAE: The Simultaneous Neural Differential-Algebraic systems of equations training Python package

Nicolas Smits¹, Laurens Lueg¹, Victor Alves¹, Carl D. Laird¹, John R. Kitchin¹, and Lorenz T. Biegler¹

¹ Department of Chemical Engineering, Carnegie Mellon University, Pittsburgh, Pennsylvania, USA

DOI: [10.xxxxxx/draft](https://doi.org/10.xxxxxx/draft)

Software

- [Review](#)
- [Repository](#)
- [Archive](#)

Editor: [Open Journals](#)

Reviewers:

- [@openjournals](#)

Submitted: 01 January 1970

Published: unpublished

License

Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License ([CC BY 4.0](#)).

Summary

Problem Statement

When modeling dynamical systems, first-principles mechanistic, constitutive, or approximate equations are often incomplete or insufficient to capture the true underlying complexity of the phenomena at play. Depending on the system, the modeling shortfall could manifest itself as for example, a reaction rate, growth term, or transfer coefficient which could depend on measured state variables via a relationship which is present yet not explicitly modeled. While domain experts may recognize these missing dynamics, formulating and validating new governing equations may be slow, or even intractable. Hybrid modeling resolves these dilemmas by leveraging existing mechanistic knowledge while embedding data-driven surrogates to improve fitting and generalization performance. In this way, hybrid models have a two-fold advantage over data-driven or purely-mechanistic approaches. Unlike data-driven models, hybrid models can enforce physical invariants such as mass, energy, or momentum balances into solution routines, allowing mappings learned from data to remain physically consistent. Simultaneously, unlike purely-mechanistic models, hybrid models are able to use valuable experimental data to inform and improve the solution of dynamical systems. The mechanistic vs data-driven tradeoff can be tuned to achieve the best performance based on modelling goals, whether that be simulation, model-based-design of experiments (MBDoE), embedded surrogate optimization etc. ([Franceschini & Macchietto, 2008](#); [Mahanty, 2023](#)).

While the predictive and extrapolative benefits of hybrid modeling are well-documented across decades of literature ([Herrera-Ruiz et al., 2025](#); [Narayanan et al., 2022](#); [Psychogios & Ungar, 1992](#); [Thompson & Kramer, 1994](#)), existing open-source software targets Ordinary Differential Equation (ODE) systems almost exclusively, with a few exceptions ([Drgona et al., 2023](#)) which solve the training problem using sequential integration. Comparatively little tooling addresses hybrid Differential-Algebraic Equation (DAE) systems, where algebraic constraints must hold alongside differential dynamics. SiNDAE fills this software gap by extending scalable hybrid modeling functionality to DAE systems, while solving the problem using the simultaneous approach for dynamic optimisation problems ([Biegler, 2007](#)).

Background

In the field of process systems modeling in science and engineering, paradigms combining data-driven components with purely mechanistic models have gained traction as promising hybrid improvements on traditional methods. The milestone approaches published in this area of research include: Physics Informed Neural Networks (PINNs) ([Raissi et al., 2019](#)), Neural Ordinary Differential Equations (NeurODEs) ([Chen et al., 2018](#)), and Universal Differential

Equations (UDEs) (Rackauckas et al., 2021). Said hybrid methods have been applied to solving ODEs in domains ranging from bioprocesses (Bang et al., 2022; Narayanan et al., 2022) to power systems (Xiao et al., 2023) to wastewater treatment (Huang et al., 2025). However, mature scientific machine learning frameworks such as torchdiffeq (Chen et al., 2018), DiffEqFlux.jl (Rackauckas et al., 2019), and diffrax (Kidger, 2022) rely on integrate-and-backpropagate mechanisms designed for ODEs. They lack native mechanisms for enforcing explicit, hard algebraic constraints during model training.

Recently, work extending hybridization to Differential Algebraic Systems of Equations (DAEs), so that explicit algebraic constraints are obeyed, has relied on sequential training or penalty loss functions. These methods use techniques such as integration and adjoint sensitivities (Kim et al., 2021), stabilization or projection onto the constraint manifold (White et al., 2023), operator splitting between the differential and algebraic subsystems (Koch et al., 2025), or penalty terms on the constraint residual (Moya & Lin, 2022). Consequently, algebraic constraints are enforced only softly or approximately. Stabilization methods keep physical invariants only approximately satisfied during simulation (White et al., 2023), and penalty formulations enforce the DAE as approximate hard constraints whose accuracy depends on hyperparameters that, chosen poorly, leave the optimization ill-conditioned or slow to converge (Moya & Lin, 2022). Furthermore, most existing DAE-focused PINN research aims to learn trajectory solutions of fully specified systems (Moya & Lin, 2022), rather than discovering unknown functional components within constrained systems.

To overcome these issues, Lueg et al. propose two novel training strategies to tackle rigorous DAE constraints enforcement, solving higher-index systems, and scalability to many trajectories (Lueg et al., 2026). The simultaneous approach employs a strategy similar to simultaneous approaches for solving purely-mechanistic DAEs by discretizing the domain using orthogonal collocation before solving the system using a capable nonlinear solver such as IPOPT (Wächter & Biegler, 2006). Said method effectively achieves constraint enforcement and higher-index system solutions, scaling to moderately sized systems. Conversely, the decomposition approach adapts training to a bi-level optimization problem, where neural network weights are updated via a sensitivity-based stochastic gradient descent outer loop, while interior iterations solve subproblems parallelized across trajectories with the neural network parameters frozen. Although the decomposition approach solves the DAE and updates the weights in alternation, it retains rigorous constraint enforcement and higher-index performance while scaling to large numbers of trajectories.

SiNDAE

SiNDAE implements the simultaneous and decomposition approaches from *A Simultaneous Approach for Training Neural DAEs* by Lueg et al. (Lueg et al., 2026), bridging the tooling and functionality gap by allowing components of DAEs to be learned subject to algebraic constraints in a scalable fashion. The package leverages Pyomo and Pyomo DAE to build the NLPs solved in both approaches, ensuring both rigorous constraint enforcement and sufficient modeling flexibility for diverse applications. SiNDAE's API design is intended to be user-friendly, mirroring widespread machine learning packages' architecture, namely scikit-learn's (Pedregosa et al., 2011) and SparkML (Meng et al., 2016) estimator oriented design. Similarly, its installation is intended to be hassle-free by alleviating the need for external binaries for the linear and nonlinear solvers required, thus streamlining adoption. Interoperability into complementary machine learning, modeling, and optimization ecosystems is facilitated through SiNDAE's export formats, enabling the dissemination of accurate hybrid models for use in downstream use-cases. Together, SiNDAE's functionality and design is intended to facilitate the training of hybrid DAE systems in a user-friendly format, with auxiliary integrations kept in mind.

Statement of need

Audience & Scientific Gap:

SiNDAE addresses a critical gap in scientific machine learning by providing a dedicated framework for hybrid Differential-Algebraic Equation (DAE) modeling. The package targets researchers and practitioners in process systems engineering, chemical engineering, and applied physics who hold mechanistic DAE models with uncharacterized or uncertain terms (e.g., complex reaction kinetics or mass transfer coefficients) and possess noisy, incomplete state measurements. While existing hybrid differential equation packages cater almost exclusively to Ordinary Differential Equations (ODEs), SiNDAE allows users to learn unknown functional components directly within index-1 and higher-index DAE systems. By enforcing governing algebraic equations as hard constraints during both training and inference, SiNDAE ensures that unobserved state variables remain physically consistent, enabling robust extrapolation to unmeasured operating regimes, a defining advantage of hybrid modeling (Herrera-Ruiz et al., 2025; Stosch et al., 2014).

Usability & Open Access

A major barrier to the adoption and open reproduction of hybrid modeling workflows is the lack of open-source software and the reliance on proprietary or complex solver installation stacks (Mahanty, 2023). Traditional dynamic optimization workflows often depend on solvers like IPOPT (Wächter & Biegler, 2006) paired with linear solvers (e.g., HSL's MA27) that require specialized licenses and compiled binaries, hindering code sharing across communal repositories. SiNDAE overcomes this hurdle by featuring a completely license-binary-free default installation powered by the POUNCE (Kitchin, 2026b) nonlinear solver and the FERAL (Kitchin, 2026a) linear solver, both distributed directly via standard Python wheels. Practitioners can execute the entire pipeline out of the box, spanning parameter pre-training and simultaneous/decomposition training to model inference, while retaining optional support for traditional cyipopt backends. Furthermore, SiNDAE adopts an estimator-oriented API architecture modeled after scikit-learn (HybridDAE and ProblemDefinition), reducing the learning curve for ML practitioners. Trained models can be exported to Equinox, ONNX, JSON, or OMLT NetworkDefinition objects (Cecon et al., 2022), allowing identified surrogates to be embedded seamlessly into downstream optimization and control workflows. Additionally, to enable expedited hybrid modeling workflows using modern AI coding agents (primarily Claude Code), a directory called sindae-skills containing an instruction file and several skill files has been included. When configured for a coding agent working on a SiNDAE project, the instruction file guides the user interaction and modelling process through latex equation rendering and transparency rules, while skill files provide design decision and implementation best practices.

Ecosystem & Dependencies

SiNDAE is built upon established scientific Python infrastructure. Symbolic model construction and Lagrange–Radau collocation on finite elements are managed via Pyomo and Pyomo.DAE (Bynum et al., 2021; Nicholson et al., 2018), while Pyomo's PyNumero interface exposes the underlying NLP structures and KKT systems for implicit differentiation. Neural network architectures, exact higher-order automatic differentiation, and outer-loop optimization updates are powered by JAX (Bradbury et al., 2018), Equinox (Kidger & Garcia, 2021), and Optax (DeepMind et al., 2020). Fundamental data manipulation, scientific routines, and visualization are supported by NumPy, SciPy, and Matplotlib. Parallelization of the decomposition strategy is enabled through the optional use of MPI for Python (L. Dalcin & Storti, 2005). Exporting and printing functionality are handled by the jax2onnx (Enpasos, 2025) and tabulate (Astanan, 2026) packages respectively. Figure 1 illustrates the resulting dependency architecture.

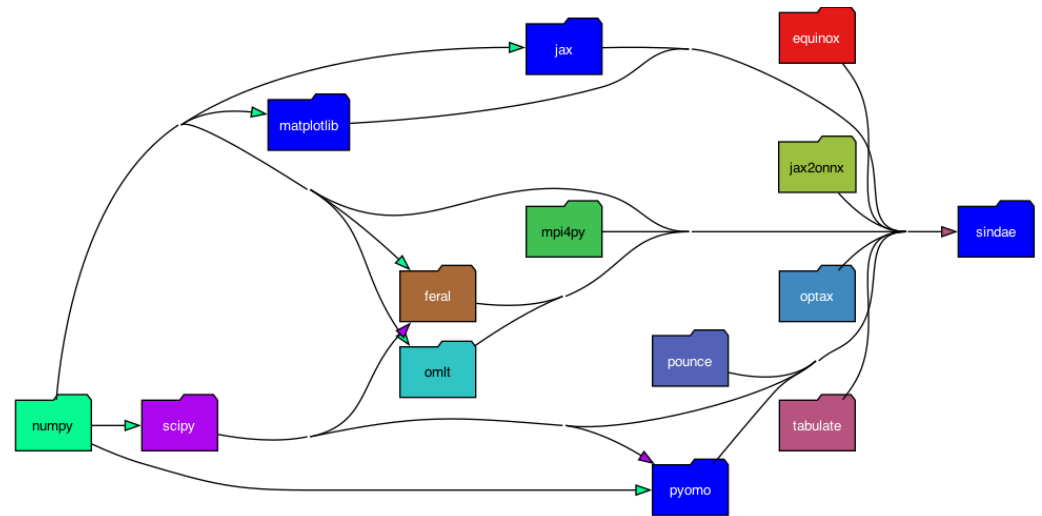


Figure 1: Dependency graph generated with pydeps showing the packages used in SiNDAE

Vignette

To illustrate SiNDAE's core capabilities and user workflow, consider a classic process engineering system: a two-tank liquid cascade operating as an index-1 Differential-Algebraic Equation (DAE), shown in Figure 2a. In this system, liquid levels $h_1(t)$ and $h_2(t)$ evolve under a constant feed rate F_{in} and inter-tank flows $q_1(t)$ and $q_2(t)$:

$$A_1 \frac{dh_1}{dt} = F_{in} - q_1, \quad A_2 \frac{dh_2}{dt} = q_1 - q_2, \quad q_2 = c_2 \sqrt{h_2}$$

In practical scenarios, physical relationships are often only partially known. Here, the outlet relation of the second tank (q_2) is explicitly governed by a known valve equation, while the inter-tank discharge law (q_1) is unknown and must be learned from noisy, sparse trajectory data of the liquid levels.

The core philosophy of SiNDAE is to eliminate the mathematical and software engineering overhead traditional in dynamic optimization. Rather than requiring users to manually formulate orthogonal collocation matrices, derive sensitivity relations, or manage complex C++/Fortran solver dependencies, SiNDAE encapsulates the entire hybrid training and inference pipeline into a high-level, estimator-oriented architecture.

To solve this problem using SiNDAE, a user only needs to perform three concise steps:

- 1. Define the Physics:** Subclass `ProblemDefinition` to specify the mechanistic DAE in Pyomo format and designate which variables enter and exit the neural surrogate (Figure 2b).
- 2. Configure and Fit:** Assemble the multi-stage training pipeline by pairing the problem with stage configuration objects. Calling `fit()` automatically handles data smoothing, supervised network pre-training, and the simultaneous non-linear program (NLP) optimization solve (Figure 2c). Users can seamlessly swap between the simultaneous and decomposition algorithms or customize neural architectures via SiNDAE's grey-box interface.
- 3. Predict and Export:** Call `predict()` on the fitted estimator to evaluate new initial conditions or operating regimes (Figure 2c). Because `predict()` re-embeds the learned surrogate into the full mechanistic DAE solver, all predicted trajectories satisfy hard algebraic constraints by construction.

As shown in Figure 2d, although the neural network $f_{NN}(h_1, h_2; \theta)$ receives no prior structural information, its learned response collapses onto the true physics ($c_1 \sqrt{h_1}$). Furthermore, predictions under unseen initial charges track reference trajectories exactly (Figure 2e). Once identified, the trained hybrid model can be directly exported to ONNX, Equinox, or OMLT formats for use in downstream surrogate optimization and control tasks.

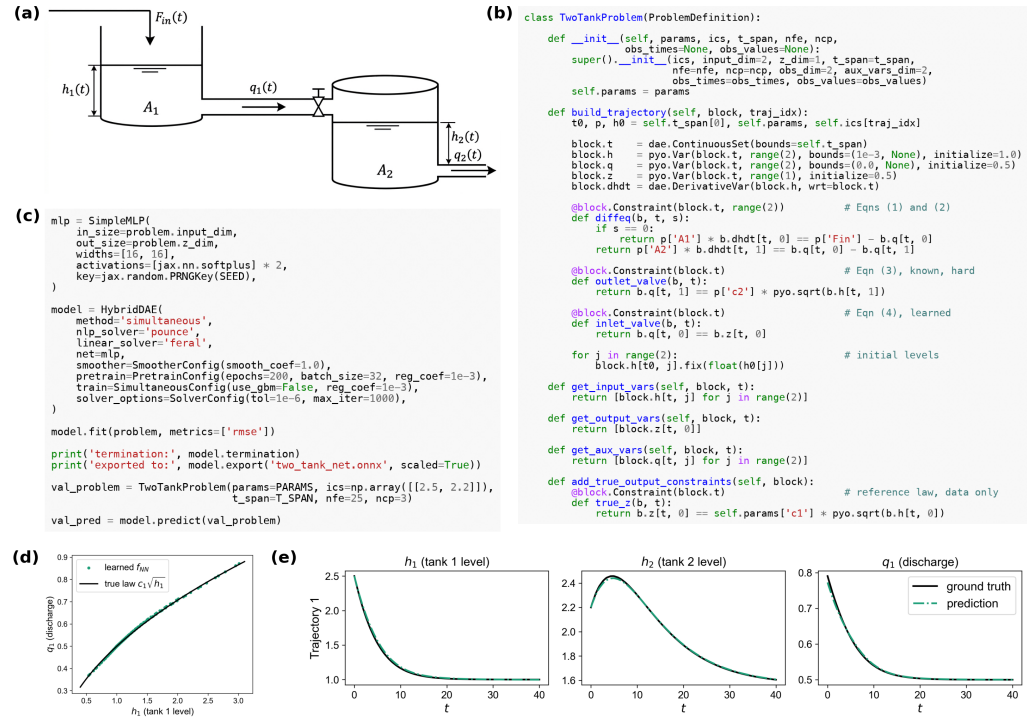


Figure 2: A complete SiNDAE workflow on a two-tank cascade DAE. (a) the physical system, with the inter-tank discharge q_1 treated as unknown; (b) system definition via a ProblemDefinition subclass; (c) estimator assembly, multi-stage fit, export, and constrained predict execution; (d) recovery of the un-modeled discharge law against the physical truth; (e) physically consistent prediction on holdout initial conditions.

Availability

SiNDAE is available on PyPI and has its source code hosted on GitHub. The documentation contains thorough descriptions of the API and functionality, as well as theoretical grounding for both the simultaneous and the decomposition methods available, instructions on how to configure a hybrid DAE system, as well as a gallery of examples.

The authors aim to supply both proper documentation to the users in the open-source software community and provide sufficient grounding in theory to enable them to design, implement, and distribute hybrid DAE models for their use-case.

AI Usage Disclosure

AI Coding assistants, primarily Claude Code, were used during the development of SiNDAE. Their role was confined to restructuring the original research code of (Lueg et al., 2026) into an installable package, migrating the solver backends, writing tests, configuring documentation, and debugging. The numerical methods, the problem formulations, and the validation criteria are the authors' own.

References

- Astanin, S. (2026). Tabulate: Pretty-print tabular data in python. In *GitHub repository*. <https://github.com/astanin/python-tabulate>; GitHub.
- Bangi, M. S. F., Kao, K., & Kwon, J. S.-I. (2022). Physics-informed neural networks for hybrid modeling of lab-scale batch fermentation for β -carotene production using *saccharomyces cerevisiae*. *Chemical Engineering Research and Design*, 179, 415–423. <https://doi.org/https://doi.org/10.1016/j.cherd.2022.01.041>
- Biegler, L. T. (2007). An overview of simultaneous strategies for dynamic optimization. *Chemical Engineering and Processing: Process Intensification*, 46(11), 1043–1053. <https://doi.org/https://doi.org/10.1016/j.cep.2006.06.021>
- Bradbury, J., Frostig, R., Hawkins, P., Johnson, M. J., Leary, C., Maclaurin, D., Necula, G., Paszke, A., VanderPlas, J., Wanderman-Milne, S., & Zhang, Q. (2018). *JAX: Composable transformations of Python+NumPy programs*. <http://github.com/google/jax>
- Bynum, M. L., Hackebeit, G. A., Hart, W. E., Laird, C. D., Nicholson, B. L., Sirola, J. D., Watson, J.-P., & Woodruff, D. L. (2021). *Pyomo—optimization modeling in python* (Third, Vol. 67). Springer Science & Business Media.
- Ceccon, F., Jalving, J., Haddad, J., Thebelt, A., Tsay, C., Laird, C. D., & Misener, R. (2022). OMLT: Optimization & machine learning toolkit. *Journal of Machine Learning Research*, 23(349), 1–8. <https://doi.org/10.48550/arXiv.2202.02414>
- Chen, R. T. Q., Rubanova, Y., Bettencourt, J., & Duvenaud, D. (2018). Neural ordinary differential equations. *Advances in Neural Information Processing Systems*, 31.
- DeepMind, Babuschkin, I., Baumli, K., Bell, A., Bhupatiraju, S., Bruce, J., Buchlovsky, P., Budden, D., Cai, T., Clark, A., Danihelka, I., Dedieu, A., Fantacci, C., Godwin, J., Jones, C., Hemsley, R., Hennigan, T., Hessel, M., Hou, S., ... Viola, F. (2020). *The DeepMind JAX Ecosystem*. <http://github.com/google-deepmind>
- Drgona, J., Tuor, A., Koch, J., Shapiro, M., King, E., & Vrabie, D. (2023). *Domain aware deep-learning algorithms integrated with scientific-computing technologies (DADAIST)* (PNNL-34895). Pacific Northwest National Laboratory.
- Enpasos. (2025). jax2onnx: Export JAX, flax NNX, flax linen, and equinox to ONNX. In *GitHub repository*. GitHub. <https://github.com/enpasos/jax2onnx>
- Franceschini, G., & Macchietto, S. (2008). Macchietto, s.: Model-based design of experiments for parameter precision: State of the art. *Chemical Engineering Science*, 63, 4846–4872. <https://doi.org/10.1016/j.ces.2007.11.034>
- Herrera-Ruiz, J. F., Fontalvo, J., & Prado-Rubio, O. A. (2025). Hybrid modeling for bioprocesses: Architectures, applications, and perspectives. *Engineering Reports*, 7(12). <https://doi.org/10.1002/eng2.70502>
- Huang, X., Kandris, K., & Katsou, E. (2025). Training stiff neural ordinary differential equations in data-driven wastewater process modelling. *Journal of Environmental Management*, 373, 123870. <https://doi.org/https://doi.org/10.1016/j.jenvman.2024.123870>
- Kidger, P. (2022). *On neural differential equations* [PhD thesis, University of Oxford]. <https://doi.org/10.48550/arXiv.2202.02435>
- Kidger, P., & Garcia, C. (2021). Equinox: Neural networks in JAX via callable PyTrees and filtered transformations. *arXiv Preprint arXiv:2111.00254*.
- Kim, S., Ji, W., Deng, S., Ma, Y., & Rackauckas, C. (2021). Stiff neural ordinary differential equations. *Chaos: An Interdisciplinary Journal of Nonlinear Science*, 31(9), 093122. <https://doi.org/10.1063/5.0060697>

- 229 Kitchin, J. R. (2026a). Feral: A pure-rust sparse symmetric indefinite linear solver with
 230 certified inertia counts. In *GitHub repository*. <https://github.com/jkitchin/feral>; GitHub.
 231 <https://github.com/jkitchin/feral>
- 232 Kitchin, J. R. (2026b). POUNCE: A pure-rust port of the ipopt interior-point NLP solver.
 233 In *GitHub repository*. <https://github.com/jkitchin/pounce>; GitHub. <https://doi.org/10.5281/zenodo.21537291>
 234 <https://doi.org/10.5281/zenodo.21537291>
- 235 Koch, J., Shapiro, M., Sharma, H., Vrabie, D., & Dragoňa, J. (2025). Learning neural differential
 236 algebraic equations via operator splitting. *2025 IEEE 64th Conference on Decision and*
 237 *Control (CDC)*, 2077–2084. <https://doi.org/10.1109/CDC57313.2025.11312239>
- 238 L. Dalcin, R. Paz, & Storti, M. (2005). MPI for python. *Journal of Parallel and Distributed*
 239 *Computing*, 65(9), 1108–1115. <https://doi.org/10.1016/j.jpdc.2005.03.010>
- 240 Lueg, L. R., Alves, V., Schicksnus, D., Kitchin, J. R., Laird, C. D., & Biegler, L. T. (2026).
 241 A simultaneous approach for training neural differential-algebraic systems of equations.
 242 *Computational Optimization and Applications*. [https://doi.org/10.1007/s10589-026-00823-](https://doi.org/10.1007/s10589-026-00823-y)
 243 [y](https://doi.org/10.1007/s10589-026-00823-y)
- 244 Mahanty, B. (2023). Hybrid modeling in bioprocess dynamics: Structural variabilities,
 245 implementation strategies, and practical challenges. *Biotechnology and Bioengineering*,
 246 120(8), 2072–2091. <https://doi.org/10.1002/bit.28503>
- 247 Meng, X., Bradley, J., Yavuz, B., Sparks, E., Venkataraman, S., Liu, D., Freeman, J., Tsai,
 248 D., Amde, M., Owen, S., Xin, D., Xin, R., Franklin, M. J., Zadeh, R., Zaharia, M., &
 249 Talwalkar, A. (2016). MLlib: Machine learning in apache spark. *J. Mach. Learn. Res.*,
 250 17(1), 1235–1241.
- 251 Moya, C., & Lin, G. (2022). DAE-PINN: A physics-informed neural network model for simulating
 252 differential algebraic equations with application to power networks. *Neural Computing and*
 253 *Applications*, 35(5), 3789–3804. <https://doi.org/10.1007/s00521-022-07886-y>
- 254 Narayanan, H., Luna, M., Sokolov, M., Butté, A., & Morbidelli, M. (2022). Hybrid models
 255 based on machine learning and an increasing degree of process knowledge: Application to
 256 cell culture processes. *Industrial & Engineering Chemistry Research*, 61(25), 8658–8672.
 257 <https://doi.org/10.1021/acs.iecr.1c04507>
- 258 Nicholson, B., Sirola, J. D., Watson, J.-P., Zavala, V. M., & Biegler, L. T. (2018). Pyomo.dae:
 259 A modeling and automatic discretization framework for optimization with differential and
 260 algebraic equations. *Mathematical Programming Computation*, 10(2), 187–223.
- 261 Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M.,
 262 Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D.,
 263 Brucher, M., Perrot, M., & Duchesnay, E. (2011). Scikit-learn: Machine learning in Python.
 264 *Journal of Machine Learning Research*, 12, 2825–2830.
- 265 Psichogios, D. C., & Ungar, L. H. (1992). A hybrid neural network-first principles approach
 266 to process modeling. *AIChE Journal*, 38(10), 1499–1511. <https://doi.org/10.1002/aic.690381003>
 267 <https://doi.org/10.1002/aic.690381003>
- 268 Rackauckas, C., Innes, M., Ma, Y., Bettencourt, J., White, L., & Dixit, V. (2019). DiffEqFlux.jl
 269 – a Julia library for neural differential equations. *arXiv Preprint arXiv:1902.02376*. <https://doi.org/10.48550/arXiv.1902.02376>
 270 <https://doi.org/10.48550/arXiv.1902.02376>
- 271 Rackauckas, C., Ma, Y., Martensen, J., Warner, C., Zubov, K., Supekar, R., Skinner, D.,
 272 Ramadhan, A., & Edelman, A. (2021). Universal differential equations for scientific machine
 273 learning. *arXiv Preprint arXiv:2001.04385*.
- 274 Raissi, M., Perdikaris, P., & Karniadakis, G. E. (2019). Physics-informed neural networks:
 275 A deep learning framework for solving forward and inverse problems involving nonlinear
 276 partial differential equations. *Journal of Computational Physics*, 378, 686–707. <https://doi.org/10.1016/j.jcp.2018.02.061>

277 [//doi.org/10.1016/j.jcp.2018.10.045](https://doi.org/10.1016/j.jcp.2018.10.045)

278 Stosch, M. von, Oliveira, R., Peres, J., & Feyo de Azevedo, S. (2014). Hybrid semi-parametric
279 modeling in process systems engineering: Past, present and future. *Computers & Chemical*
280 *Engineering*, 60, 86–101. <https://doi.org/10.1016/j.compchemeng.2013.08.008>

281 Thompson, M. L., & Kramer, M. A. (1994). Modeling chemical processes using prior
282 knowledge and neural networks. *AIChE Journal*, 40(8), 1328–1340. <https://doi.org/https://doi.org/10.1002/aic.690400806>

284 Wächter, A., & Biegler, L. T. (2006). On the implementation of an interior-point filter
285 line-search algorithm for large-scale nonlinear programming. *Mathematical Programming*,
286 106(1), 25–57.

287 White, A., Kilbertus, N., Gelbrecht, M., & Boers, N. (2023). Stabilized neural differential
288 equations for learning dynamics with explicit constraints. *Advances in Neural Information*
289 *Processing Systems*, 36, 12929–12950. <https://doi.org/10.48550/arXiv.2306.09739>

290 Xiao, T., Chen, Y., Huang, S., He, T., & Guan, H. (2023). Feasibility study of neural ODE
291 and DAE modules for power system dynamic component modeling. *IEEE Transactions on*
292 *Power Systems*, 38(3), 2666–2678. <https://doi.org/10.1109/tpwrs.2022.3194570>

DRAFT